

Atelier 13 – JavaScript (Navigateur 4)

matisse

kra

bts sio

sommaire

1. Objectifs du TP	1
2. Concepts Théoriques Abordés	1
A. Les Événements de Souris	1
B. Coordonnées : Client vs Page	2
C. Gestion complexe du Survol (mouseover vs mouseenter)	2
D. Délégation et Clavier	2
3. Exercices Pratiques (Travaux Dirigés)	2
Conclusion	6

1. Objectifs du TP

L'objectif principal de cet atelier est de maîtriser les événements d'interface utilisateur (UI Events) pour créer des interactions riches et robustes. Il vise spécifiquement à :

- Comprendre en détail le fonctionnement et les propriétés des événements de la souris (clic, survol, déplacement) au-delà de l'usage basique.
- Apprendre à gérer les coordonnées du curseur par rapport à la fenêtre ou au document.
- Distinguer les subtilités entre les différents types de survol (`mouseover` vs `mouseenter`) pour éviter les erreurs de traitement lors de la navigation entre éléments parents et enfants.
- Optimiser les performances via la délégation d'événements.
- Introduire la gestion des événements clavier (`keydown`, `keyup`).

2. Concepts Théoriques Abordés

A. Les Événements de Souris

Le cours détaille la mécanique précise des interactions souris :

- Séquence du Clic : Un événement `click` n'est validé que si l'utilisateur enfonce (`mousedown`) puis relâche (`mouseup`) le bouton gauche sur le même élément.

matisse kra
bts sio

- Propriétés : L'objet événement permet d'identifier quel bouton a été pressé via `event.button` (0 pour gauche, 2 pour droit) et si des touches modificatrices (Shift, Alt, Ctrl) étaient actives.
- Sélection de texte : Le `mousedown` déclenche par défaut la sélection de texte. Il est souvent nécessaire d'annuler ce comportement (via `return false`) pour des interfaces interactives (comme une liste d'éléments).

B. Coordonnées : Client vs Page

Une distinction essentielle est faite pour localiser la souris :

- `clientX/Y` : Coordonnées relatives à la fenêtre visible (viewport). Si on défile (scroll), ces valeurs ne changent pas pour un point fixe de l'écran.
- `pageX/Y` : Coordonnées relatives au document entier. Elles prennent en compte le défilement de la page.

C. Gestion complexe du Survol (mouseover vs mouseenter)

C'est un point central de l'atelier pour éviter les bugs d'interface :

- `mouseover` / `mouseout` : Ces événements se propagent ("bubble"). Le problème est qu'ils se déclenchent même lorsque la souris passe d'un élément parent à son propre enfant, ce qui peut causer des clignotements indésirables. On utilise `relatedTarget` pour savoir d'où vient ou où va la souris.
- `mouseenter` / `mouseleave` : Ces événements ne se propagent pas et ignorent les transitions internes (parent vers enfant). Ils sont idéaux pour gérer l'entrée/sortie d'une zone globale sans "bruit".

D. Délégation et Clavier

- Délégation : Pour gérer des événements sur un grand tableau, on place un seul écouteur sur le parent `<table>` plutôt que sur chaque cellule `<td>`. Comme `mouseenter` ne remonte pas (pas de bubbling), on doit utiliser `mouseover` et filtrer manuellement les cibles.
- Clavier : Distinction entre `event.code` (la touche physique, ex: "KeyZ") et `event.key` (le caractère généré selon la langue, ex: "z").

3. Exercices Pratiques (Travaux Dirigés)

L'atelier met en pratique ces notions à travers quatre exercices principaux :

1. Liste Sélectionnable : Création d'une liste type "gestionnaire de fichiers" où le clic sélectionne un item (en ajoutant une classe CSS). Si on maintient `Ctrl` (ou `Cmd` sur Mac), on peut en sélectionner plusieurs. Il faut aussi empêcher le texte d'être surligné (sélection native du navigateur).

matisse kra
bts sio

le script

```
<script>
  const liste = document.getElementById('ul');

  // Empêcher la sélection du texte lors du double-clic (mousedown) [cite: 80, 87]
  liste.onmousedown = function() {
    return false;
  };

  liste.onclick = function(event) {
    // Si l'élément cliqué n'est pas un LI (ex: on clique un peu à côté), on ignore
    if (event.target.tagName !== "LI") return;

    // Vérifier si Ctrl (Windows) ou Cmd (Mac) est enfoncé [cite: 118, 119]
    if (event.ctrlKey || event.metaKey) {
      // Mode "Sélection multiple" : on inverse juste la classe de l'élément cliqué
      event.target.classList.toggle('selected');
    } else {
      // Mode "Sélection unique"
      // 1. On retire la classe 'selected' de tous les éléments
      let items = liste.querySelectorAll('li');
      for (let item of items) {
        item.classList.remove('selected');
      }
      // 2. On l'ajoute sur celui qu'on vient de cliquer
      event.target.classList.add('selected');
    }
  }
</script>
```

2. Tableau de Bagua (Délégation) : Mise en surbrillance d'une cellule <td> au survol. L'objectif est d'utiliser un seul gestionnaire sur la table (délégation) tout en gérant correctement les transitions complexes (filtrage via relatedTarget pour ignorer les mouvements internes à la cellule).

On utilise mouseover et mouseout avec la propriété relatedTarget
le script

```
<!-- /Volumes/LaCie/bts sio 1/javascript/liste (1).html -->
let table = document.getElementById('pagua-table'); // Supposons que ta table a cet ID
let selectedTd;

table.onmouseover = function(event) {
    // On cherche le TD le plus proche de la cible (au cas où on survole une balise dans le TD)
    let target = event.target.closest('td');

    // Si on n'est pas sur un TD ou si on est hors de notre table, on ignore
    if (!target) return;
    if (!table.contains(target)) return;

    // Optimisation : Si on est déjà sur ce TD, ne rien faire
    if (selectedTd == target) return;

    highlight(target);
};

table.onmouseout = function(event) {
    // Si on quitte la table, ou qu'on va vers un élément qui est DANS le TD actuel
    // alors on ne doit pas désélectionner. relatedTarget est l'élément vers lequel on va.
    let target = event.target.closest('td');
    let related = event.relatedTarget;

    // Si on va vers un élément qui est à l'intérieur du TD actuel, on ignore l'événement
    if (target && related && target.contains(related)) {
        return;
    }

    unhighlight(selectedTd);
};

function highlight(td) {
    if (selectedTd) {
        unhighlight(selectedTd); // Enlever l'ancien s'il existe
    }
    selectedTd = td;
    selectedTd.classList.add('highlight'); // Ajouter une classe CSS pour la couleur
}

function unhighlight(td) {
    if (td) {
        td.classList.remove('highlight');
        selectedTd = null;
    }
}
```

3. Info-bulle "Intelligente" : Affichage d'une infobulle au survol d'éléments imbriqués. Le script doit déterminer l'élément le plus profond possédant l'attribut data-tooltip et afficher uniquement cette infobulle.
- le script

```
<script>
Click to collapse the range. uteur sur le document entier (délégation globale)
document.onmouseover = function(event) {
  // On cherche l'élément avec data-tooltip le plus proche de la souris
  let target = event.target.closest('[data-tooltip]');

  if (!target) return; // Pas de tooltip ici

  // Création de l'infobulle (div)
  let tooltipElem = document.createElement('div');
  tooltipElem.className = 'tooltip';
  tooltipElem.innerHTML = target.dataset.tooltip;
  document.body.append(tooltipElem);

  // Positionnement (mathématiques pour centrer et placer au-dessus)
  let coords = target.getBoundingClientRect();

  let left = coords.left + (target.offsetWidth - tooltipElem.offsetWidth) / 2;
  if (left < 0) left = 0; // Ne pas sortir à gauche

  let top = coords.top - tooltipElem.offsetHeight - 5;
  if (top < 0) { // Si pas de place au-dessus, on met en dessous
    top = coords.top + target.offsetHeight + 5;
  }

  tooltipElem.style.left = left + 'px';
  tooltipElem.style.top = top + 'px';
};

document.onmouseout = function(e) {
  let tooltip = document.querySelector('.tooltip');
  if (tooltip) {
    tooltip.remove(); // On supprime l'infobulle quand la souris part
  }
};
</script>
```

4. Champ Téléphone (Filtrage) : Création d'un champ input qui n'accepte que les chiffres et symboles téléphoniques. Utilisation de `keydown` pour bloquer les touches invalides tout en autorisant explicitement les touches de navigation (Backspace, ArrowLeft, etc.).
- ici, on utilise l'événement `keydown` car `keypress` est déprécié et `input` gère mal le blocage préventif.
- le script

```
<script>
function checkPhoneKey(key) {
  // Liste des touches spéciales autorisées (navigation, effacement)
  const allowedKeys = ['Backspace', 'Delete', 'ArrowLeft', 'ArrowRight', 'Tab'];

  // Liste des caractères autorisés
  const allowedChars = ['+', '(', ')', '-'];

  // Autoriser si c'est un chiffre (0-9)
  if (key >= '0' && key <= '9') return true;

  // Autoriser si c'est une touche spéciale
  if (allowedKeys.includes(key)) return true;

  // Autoriser si c'est un caractère spécial accepté
  if (allowedChars.includes(key)) return true;

  // Sinon, on refuse
  return false;
}

// Utilisation sur l'input HTML :
// <input onkeydown="return checkPhoneKey(event.key)" placeholder="Téléphone">
</script>
```

Conclusion

Cet atelier permet de maîtriser la différence subtile mais cruciale entre les différents événements de la souris, particulièrement pour éviter les bugs de clignotement lors des survols d'éléments complexes. Il introduit également la logique de délégation d'événements, une technique essentielle pour optimiser les performances en JavaScript en réduisant le nombre d'écouteurs (listeners).